PV-Triage — Simple Visual Guide

What the 3 screens show, how they work, and all files used

What is PV-Triage? It is a clinical safety helper that reads a medicine side-effect report, checks official rules in PDFs, and **recommends** whether it needs fast **15-day reporting** or **human doctor (QPPV) review**. It never submits a report on its own.

`localhost:16686`

1. Jaeger (Trace Viewer)

Shows a timeline of every request and background step so you can see speeds and spot errors.

`localhost:3000`

2. Grafana (Health Monitor)

Shows live charts of uptime, speed (ms), queue depth, cache hits, errors, and CPU/RAM.

`localhost:8000`

3. PV-Triage Web App

The main clinical screen where you submit an adverse event report and see the AI verdict.

Where New Files Were Added

1. This New PDF Guide: Saved at `docs/PV_Triage_Visual_Guide.pdf` (inside the `docs/` folder with `docs/report.pdf`).
2. Added / Updated UI Files in Workspace: `design-system.md` (theme tokens), `api/web/vendor/gsap.min.js` & `lenis.min.js` (animations), `api/web/index.html`, `app.js`, `styles.css` (cockpit UI), and `docs/screens/*.png` + `scripts/capture_screens.py`.

What All Project Files We Used (Simple File Map)

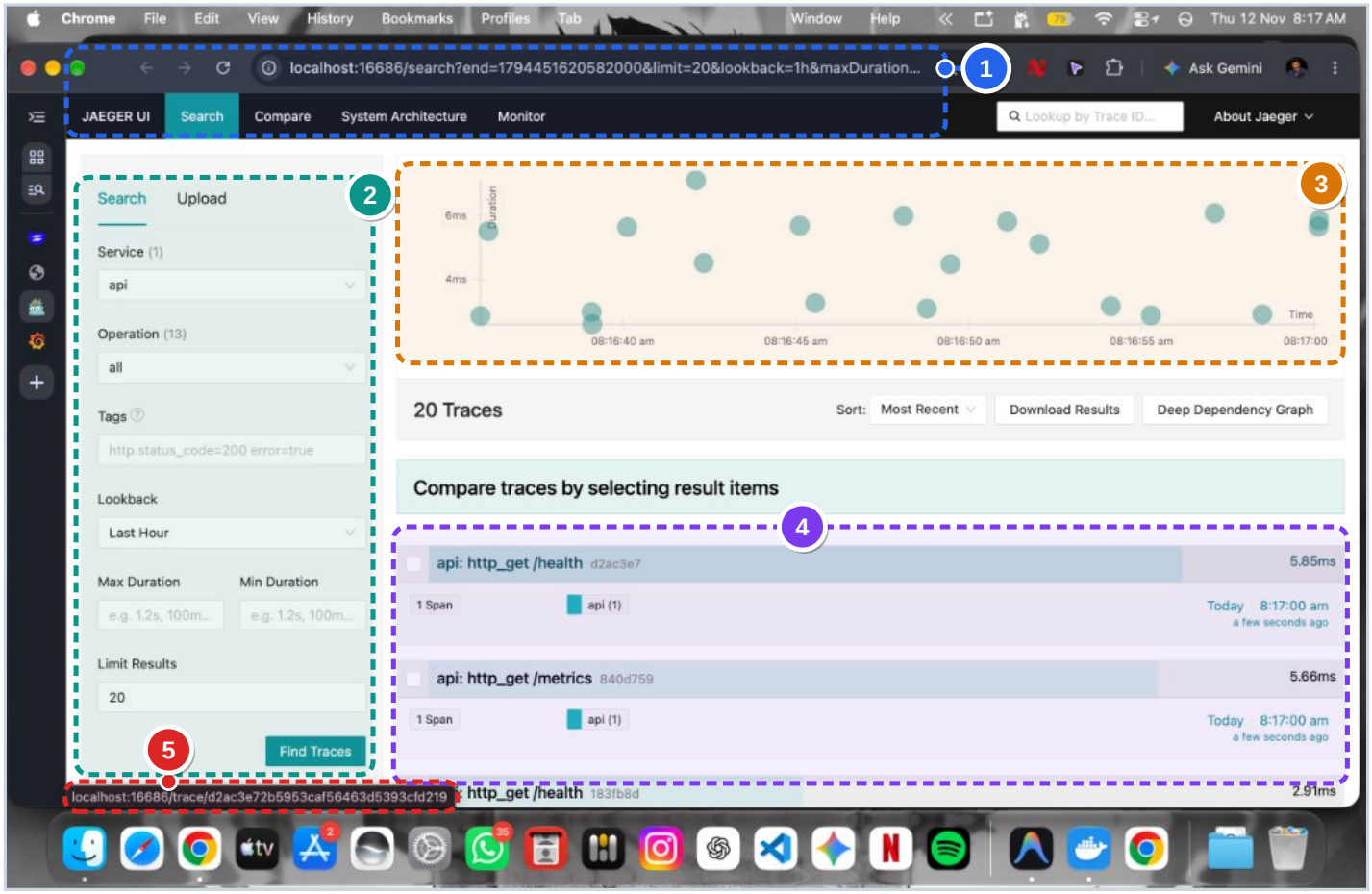
Part	Files Used	What They Do (Simple)
Web UI	`api/web/index.html`, `app.js`, `styles.css`, `vendor/*`	Draws the intake form, benchmark buttons, decision cards, and theme switch.
REST API	`api/main.py`, `api/models.py`, `api/logging_setup.py`	Validates inputs, quarantines bad forms, caches runs, serves `/metrics`.
Queue & Worker	`worker/main.py`, `worker/tasks.py`	Picks up triage jobs from Redis RQ in the background so the UI stays fast.
AI Agents	`agents/graph.py`, `run.py`, `tools.py`, `state.py`	Assessor evaluates seriousness & label; Verifier checks quotes 100% verbatim.
LLM & Guardrails	`stub_llm/main.py`, `llm/*`, `guardrails/*`, `prompts/*`	Offline Stub LLM, circuit breaker, prompt injection filter, and prompts.
Corpus & Search	`pipeline/ingest.py`, `retrieval.py`, `synonyms.json`, `corpus/*.pdf`	Loads 3 official PDFs (IPC, NDCT 2019, Lamotrigine PI) into Chroma vector search.
Monitoring	`shared/telemetry.py`, `metrics.py`, `infra/prometheus.yml`, `infra/grafana/*`	Sends traces to Jaeger, exposes metrics to Prometheus, builds Grafana charts.
Docker Setup	`docker-compose.yml`, `.env.example`, `Makefile`	Starts all 9 services together (API, Worker, Redis, Stub LLM, Jaeger, Prometheus, Grafana, cAdvisor, Ingest).

How the 3 Screens Work Together: (1) Submit a case in the **Web App** (`:8000`) → **(2)** API & Worker run AI + PDF checks and send timing traces to **Jaeger** (`:16686`) → **(3)** Prometheus collects live numbers and displays system health in **Grafana** (`:3000`).

Screen 1: Jaeger UI — Distributed Trace Viewer

URL: http://localhost:16686 • Shows request timelines & speeds

What this represents & does: Jaeger records a **trace** (timeline) for every API call and worker run. It lets you check exact millisecond speeds and see which step ran or failed.



1 Jaeger URL & Top Tabs

Open at `localhost:16686`. Switch between **Search**, **Compare** two runs, or the **System Architecture** view.

2 Search & Filter Panel

Filter traces by service (`api` or `worker`), operation (`all`), time (`Last Hour`), and max results (`20`).

3 Duration Scatter Plot

Each teal dot is one request plotted over time. Height shows speed — all calls here took **2–7 ms**.

4 20 Captured Traces List

Lists individual API calls like `http_get /health (5.85ms)` and `/metrics (5.66ms)` from live health polling.

5 Clickable Trace Link Preview

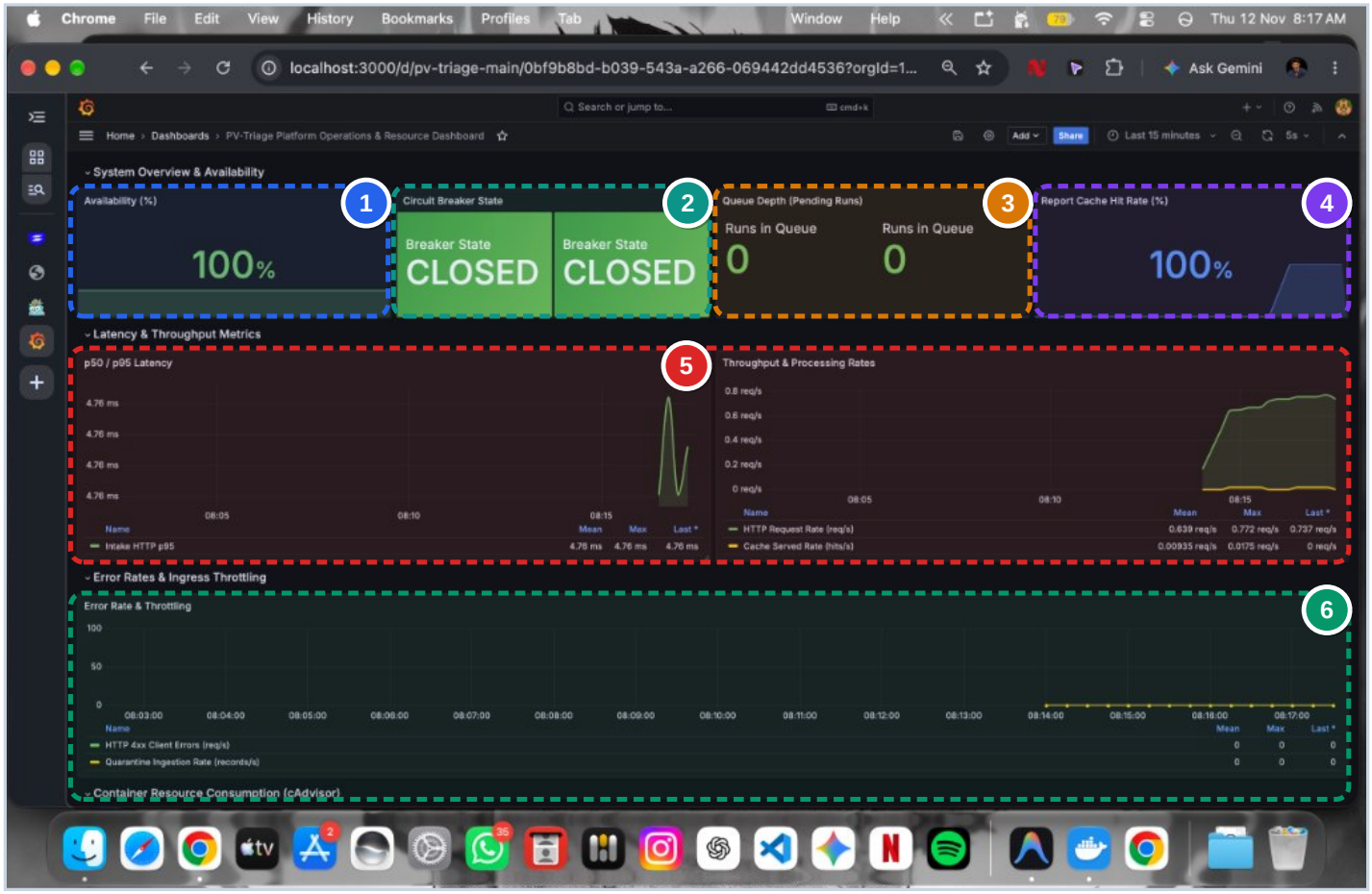
Hovering shows the direct trace URL (`.../trace/d2ac3e72...`). Click any row to see agent & tool child spans inside.

Files used for this screen: `docker-compose.yml` (jaeger ports 16686/4317/4318) • `shared/telemetry.py` (OTLP export & SQLite trace save) • `api/main.py` (HTTP spans) • `worker/tasks.py` (agent & tool spans)

Screen 2: Grafana — Live Health & Metrics Dashboard

URL: http://localhost:3000 • Live data collected by Prometheus every 3 seconds

What this represents & does: This is the **PV-Triage Platform Operations & Resource Dashboard**. Prometheus collects numbers from the API, Worker, and Docker, and Grafana shows live health graphs.



1 Availability (100%)

Shows uptime without `5xx` crashes. **100%** means the platform is completely healthy.

2 Circuit Breaker State (CLOSED)

Green **CLOSED** means the LLM connection works. If the LLM fails, it turns red **OPEN** to protect the worker.

3 Queue Depth (0 Pending)

Shows reports waiting in the Redis RQ queue. **0** means background workers are caught up.

4 Report Cache Hit Rate (100%)

Shows repeat reports served instantly from Redis cache without calling the LLM again.

5 Latency (4.76 ms) & Throughput

Left chart checks API speed vs. the 5s limit (**4.76 ms** here). Right chart shows requests/sec (`0.737 req/s`).

6 Errors & Container Resources

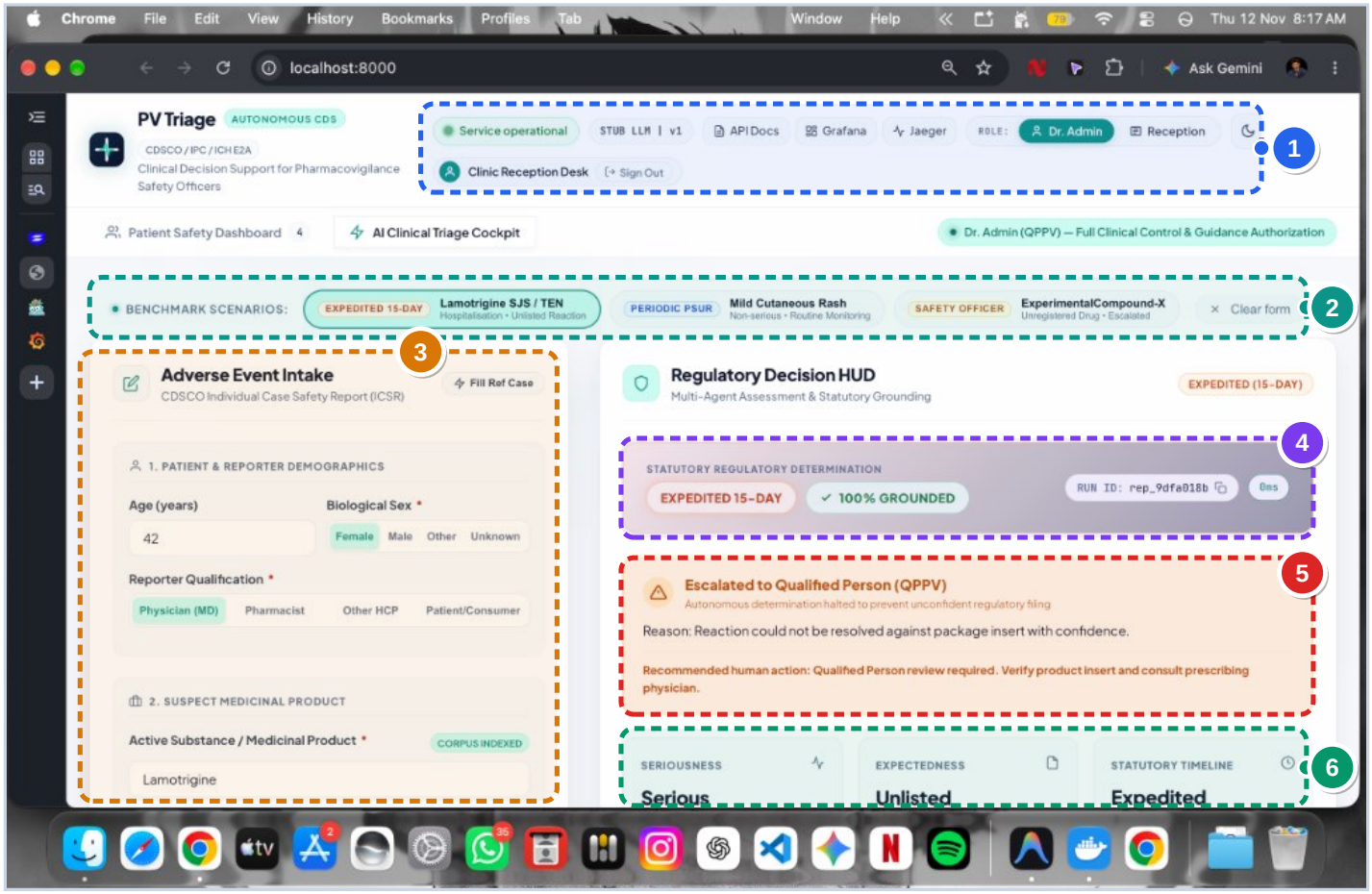
Tracks HTTP 4xx/429/quarantine rates (all **0**), plus Docker CPU & RAM right below via **cAdvisor**.

Files used for this screen: `docker-compose.yml` (prometheus:9090, grafana:3000, cadvisor:8080) • `infra/prometheus.yml` • `infra/grafana/dashboards/pv_triage_dashboard.json` • `infra/grafana/provisioning/*` • `shared/metrics.py` • `api/main.py` (`/metrics`)

Screen 3: PV-Triage Web App — Clinical Triage Cockpit

URL: http://localhost:8000 • Doctor intake form & AI regulatory decision screen

What this represents & does: This is the **main PV-Triage user interface**. A safety officer submits a drug side-effect case on the left, and the AI shows its grounded recommendation on the right.



1 Status Bar & Tool Links
Shows `Service operational`, `STUB LLM | v1`, quick buttons for **API Docs / Grafana / Jaeger**, and role toggle (`Dr. Admin`).

2 Benchmark Scenarios Bar
1-click sample buttons: **Lamotrigine SJS/TEN** (15-day expedited), **Mild Rash** (periodic), **ExperimentalCompound-X** (escalated).

3 Adverse Event Intake Form
Captures Age (42), Sex (`Female`), Reporter (`Physician`), and Drug (`Lamotrigine • CORPUS INDEXED`).

4 Regulatory Verdict Banner
Shows **EXPEDITED 15-DAY**, **100% GROUNDED** (quotes verified in corpus PDFs), `RUN ID: rep_9dfa018b`, and `0ms` (Redis cache hit).

5 QPPV Human Escalation Box
Alerts a Qualified Person (QPPV) when human review is needed. *(Note: visible here due to a GSAP animation reset in `app.js` even though grounded.)*

6 3 Decision Summary Cards
Splits the verdict into 3 simple checks: **Seriousness** (`Serious`), **Expectedness** (`Unlisted`), and **Timeline** (`Expedited`).

Files used for this screen: `api/web/index.html`, `app.js`, `styles.css`, `vendor/*` • `api/main.py` & `models.py` • `agents/graph.py` & `tools.py` • `pipeline/retrieval.py` • `corpus/*.pdf` • `stub_llm/main.py`